LDX3 — DIRECTOR+ WORKSHOP 2026

Beyond the *hype*.

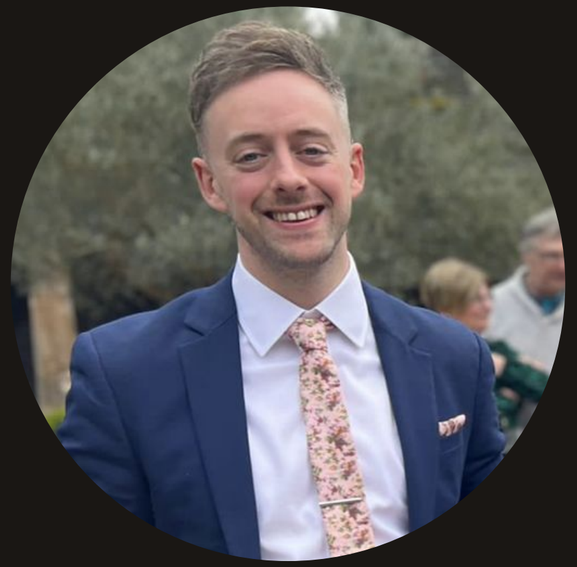
Adopting and measuring AI tools safely in regulated, high-velocity engineering teams.

Luke Briscoe
Engineering Director
Monzo

JUNE 2026
LONDON

Chatham House Rules

You’re free to use information from discussions today, but don’t reveal who made any particular comment..



Hello!

LUKE BRISCOE | MONZO

ENGINEERING DIRECTOR; PLATFORM & SECURITY

DEVELOPER EXPERIENCE & AI TOOLING

Two questions we'll explore together

01

How do you design *guardrails* that catch AI debt — without killing velocity and properly considering regulation?

SYSTEM DESIGN

02

How do you measure *real Return on Investment (ROI)* beyond lines of code and PR throughput?

MEANINGFUL MEASUREMENT

01

What the data tells us.

93%

of engineers now use AI coding tools at least monthly.

The adoption argument is over. The tools are here, and we're using them.

4.7 *hrs*

saved per week — what daily users tell us AI gives them back.

Self-reports skew high. But even halved, it's a meaningful slice of every engineer’s week. But where does that saved time go instead?

8-11%

measured PR throughput increase.

The number that breaks the simple narrative. Monthly active use is up to 93% — but throughput moved only slightly?

30.8%

of daily merged code is now AI-authored — up from 24% last quarter

Implications for review, ownership, incident response — and what your auditors or regulators ask about in twelve months.

+6.8 PP vs. Q4 2025

+2 *pp*

Change Failure Rate increase — with some orgs shipping 50% more defects.

Are your guardrails keeping up with AI adoption?

MICROSOFT RESEARCH · HOUCK ET AL.

14%

of a developer's day is actually writing code.

AI optimises this slice.

Meetings, reviews, design, debugging-in-your-head, waiting on CI — untouched by your new tools.

THE OTHER 86%?

01 THE DATA

Three questions *for the room.*

01	Does an *8–11% PR throughput increase* match your experience — or are you an outlier?	THROUGHPUT
02	If your developers really are saving four-plus hours a week, *where* is that time going instead?	TIME
03	Has your *Change Failure Rate* moved since AI adoption increased?	QUALITY

Introductions & discussion.

93%	Engineers using AI coding tools at least monthly.
4.7*hrs*	Saved per week — what daily users self-report.
8–11%	Actual PR throughput increase over 16 months
30.8%	Of merged code is AI-authored
+2*pp*	Change Failure Rate increase
14%	Of an engineer's day is in-IDE coding

01 Does 8–11% PR throughput increase match your experience — or are you an outlier?

02 If your developers really are saving four-plus hours a week, where is that time going instead?

03 Has your Change Failure Rate moved since AI adoption increased?

02

System design.

Guardrails that don't kill velocity.

Question one:

How do you design guardrails that catch AI debt — without killing velocity and properly considering regulation?

Regulation defines the guardrails. *It doesn't veto.*

Compliance is not a stop sign — it's a specification for what good looks like.

Data flows, controls, auditability.

Different regimes ask different questions; flexibility in your design and adoption is key.

The org that ships fastest under regulation is the one that designs *for* it from day one.

Enable AI tools *with controls.* Don't block them.

Banning tools doesn't reduce risk. It moves it somewhere you can't see.

Block AI, and engineers will route around — personal subscriptions, shadow accounts, undisclosed usage.

You don't gain safety. You lose visibility, which is strictly worse.

The constraints sit on data flow and capability, not existence.

The risk is the *data* you share — not the model.

In regulated environments, the tool itself isn't necessarily the threat surface.

***What it can reach is** — so treat data governance as the risk to manage.*

Every consequential decision auditable — with human-in-the-loop where blast radius justifies it.

Continuous monitoring of what gets ingested into models.

Engineers and non-engineers need *different guardrails.*

Years of secure-dev muscle memory don't transfer to non-engineers.

PR review is the wrong place to absorb non-engineer risk.

Solve at the platform: sandboxing, variable friction, isolated envs

Reduce the blast radius of what non-engineers can build and ship.

If you’re not building the models, *own the interface.*

Treat model providers like database infrastructure — critical, but swappable.

An LLM gateway between you and providers gives resilience when one has an outage — important for resiliency (and regulation!).

The investment compounds across model generations. The model itself depreciates in months.

Centralise *telemetry* and *evaluation.*

Direct team-to-provider connections produce fragmentation that's expensive to undo.

One interface lets you run evals across models, see where data flows, route changes transparently.

Six scenarios. Mark each one.

01 **Production data, AI query.**
A developer needs data from prod to pass into Claude. Are there controls — or is it down to individual judgement?

02 **Data breach at a major provider.**
PII may only allowed in certain tools — but do you understand your *real* exposure in there's a breach?

03 **Regulator asks; you have 48 hours to respond.**
Account for what AI tools can access, what they can't, and how it's enforced. Can you reply in time with confidence?

04 **Personal ChatGPT + pasting sensitive data.**
Engineer pastes a payments stack trace into a personal account to help debug. Would you know?

05 **Provider outage.**
OpenAI / Anthropic is down. Your engineers can't work effectively. How fast can you reroute? Do you own that layer?

06 **New, better model lands.**
Teams are desperate for access. Is there a process for evaluating models and shipping them quickly — and how do you do that?

✓ We've solved this.

Controls are designed, deployed, and somebody owns them.

! We're exposed here.

We know the gap — and we know we haven't closed it yet.

? Haven't faced it / not sure.

No incident yet / no data either way / just don't know!

Commitment time:

What do you intend to do when you're back at work now you've explored the guardrails and principles?

SECTION THREE · 50–85 MIN

03

Measurement.

Crafting an ROI scorecard you can build on.

Question two:

How do you measure real Return on Investment (ROI) beyond lines of code and PR throughput?

The numbers that go up *while quality goes down.*

WHAT LOOKS GREAT ON A DASHBOARD	WHAT CAN QUIETLY DEGRADE UNDERNEATH
↗ % of total code written by AI	↘ Time to review PRs
↗ PRs merged	↘ Defect rate
↗ Time-to-first-commit	↘ Developer experience scores
↗ AI tool adoption %	↘ Long-tail incident MTTR

Start by asking:

Why are we even doing this....

And as an extension of that...

What does your company (and your board) care about?

A framework for thinking about measurement

A framework for thinking about measurement

Velocity

A framework for thinking about measurement

Velocity

Quality

A framework for thinking about measurement

Velocity

Quality

Developer Experience

A framework for thinking about measurement

Velocity

Quality

Developer Experience

Economic efficiency

Your *AI ROI Scorecard.* 3–4 metrics per column.

* MARK THE KEY METRICS YOUR BOARD WILL BE INTERESTED IN!

01 VELOCITY	*02* QUALITY	*03* DEVELOPER EXPERIENCE	*04* ECONOMIC EFFICIENCY

SECTION FOUR · 85–90 MIN

04 Commitments

YOU’VE SPENT ALMOST 90 MINS IN HERE — WHAT’S GOING TO CHANGE WHEN YOU LEAVE?

Commitment card.

WHAT I INTEND TO DO NOW I’VE EXPLORED THE GUARDRAILS & PRINCIPLES… _____

THE KEY NEW METRICS I’LL USE TO TRACK ROI ARE… _____

YOUR ACCOUNTABILITY PARTNER & DATE TO FOLLOW UP ON _____

Thank you!

lukebriscoe.com